Appendix (Chapter 16) — AI for Your Career

Last reviewed: September 2026. Platform menus (LinkedIn, Upwork, Fiverr — a professional network and two freelance marketplaces) and chatbot (chat-style AI assistant) features change often; the prompt patterns and honesty rules here do not.

The rest of the book teaches how AI systems work; this appendix is about using them for yourself — a stronger resume, portfolio, cover letter and LinkedIn profile, better networking messages, filtering jobs, rehearsing interviews. It is mostly **prompt patterns** (reusable fill-in-the-blank instructions) to copy, fill and iterate on, under one rule above all: **the AI may improve how you say things; it may never change what is true.**

Prompting technique (role, context, few-shot examples — worked examples in the prompt — iteration, fixing bad outputs) is taught in Chapter 5; here it is applied to career documents.

What you'll learn

- Using a chatbot as a writing partner without letting it invent your history (the *fact sheet* method, with a runnable checker)
- One master prompt pattern for every career document
- Portfolio: four core sections, challenge → solution → result write-ups, bullet expansion, a consistent voice
- Resume: how it differs from a portfolio, section prompts, bullet templates, tailoring to a job description and applicant tracking systems (with a runnable keyword-gap script)
- Cover letters: structure, company-specific drafts, your own voice, length, common mistakes
- Networking and job-search emails: cold outreach, follow-ups, referrals, thank-you notes, informational interviews
- LinkedIn: headline, About, Experience, Featured, Skills, media and links, staying active
- Freelancing: marketplaces, winning proposals, link hubs and personal sites
- Job-hunt automation and keywords (search optimisation) across portfolio, resume and LinkedIn
- Interview preparation: technical practice, behavioural answers with STAR (§16.10), follow-up and reflection
- Personal brand, consistency across platforms, reputation management, keeping everything current
- Ethics: fabrication, disclosure, privacy, interview rules

Prerequisites. Any chat assistant (ChatGPT, Claude, Gemini, Copilot…). The two scripts in `code/` run offline with the book's Python environment.

16.1 Ground rules: a writing partner, not an author

What it is

A chat assistant is a **large language model (LLM)** — trained on huge amounts of text to predict likely next words, so it writes fluent, context-aware prose (Chapter 4). Built on the **transformer** (Chapter 3), it reads your whole request and the conversation so far, so it can use details given earlier in the chat. It doesn't remember other chats unless the product has a memory feature, and very long chats can exceed its **context window** (the most text it can read at once) — so paste your fact sheet again for each new document.

For career writing it plays four roles:

Role	What you ask for	Example
Brainstormer	Options when you are stuck	"Give me five ways to open my portfolio's About page."
Editor	Grammar, structure, tone	"Tighten this paragraph; keep every fact."
Rewriter	Alternatives aimed at a reader	"Rephrase this for a hiring manager in fintech."
Coach	Critique and practice	"Act as an interviewer and grill me on this project."

A **prompt** is the instruction you give; **prompt engineering** is shaping it — context, facts, audience, format, limits — so the output fits. "Write a bio for a developer" gets a generic bio; the same prompt with your languages, years, target industry and length gets something near usable.

Why it matters

Career documents are short, high-stakes and re-tailored constantly; AI makes tailoring cheap. But an LLM has no idea what you did: missing details get filled with *plausible* ones — a team size, a percentage, a tool you never used. That is **hallucination** (Chapter 4), and on a resume it is a false claim with your name on it.

How it works: the fact-sheet method

1. **Write a fact sheet first** — a private list of true, checkable facts: roles, dates, tools, projects, and every number you can defend (with its source).
2. **Give the model only those facts** and forbid additions; ask for `[ASK ME]` where something is missing.
3. **Check the draft against the fact sheet** — every number, tool, title and employer must trace back to it.
4. **Edit into your own voice** — read aloud; if you wouldn't say it in an interview, change it.

`code/claim_check.py` automates step 3 with **regular expressions** (compact text patterns such as "any run of digits"): any number or capitalised name in the draft that isn't in your fact sheet is flagged.

```python
# code/claim_check.py (core)
def build_prompt(facts, task, audience, limit):
    """The master pattern: role + context + facts + task + constraints + format."""
    fact_lines = "\n".join(f"- {a}" for a in facts["achievements"])
    return ("You are an experienced technical recruiter and editor.\n"
            f"Context: I am a {facts['role']} with {facts['years']} years at "
            f"{facts['employer']}. Tools: {', '.join(facts['tools'])}.\n"
            f"Verified achievements (use ONLY these facts):\n{fact_lines}\n"
            f"Task: {task}\nAudience: {audience}\nLength: {limit}\n"
            "Rules: do not invent numbers, tools, titles or employers. "
            "If something is missing, write [ASK ME] instead of guessing.")

def check_draft(draft, facts):
    numbers, names = allowed_tokens(facts)          # everything in the fact sheet
    problems = []
    for n in re.findall(r"\d+(?:\.\d+)?%?", draft):
        if n not in numbers:
            problems.append(f"number '{n}' is not in your fact sheet")
    for m in re.finditer(r"(?<![.!?]\s)(?<!^)\b([A-Z][A-Za-z+#]+)", draft):
        if m.group(1).lower() not in names:
            problems.append(f"name/tool '{m.group(1)}' is not in your fact sheet")
    return problems
# ... a FakeLLM returns a draft that embellishes on purpose
```

Output (prompt part abbreviated):

```
DRAFT: Data Analyst with 2 years of experience who automated weekly reporting (6 hours to 1 hour) and built a Tableau dashboard for 40 ward managers. Led a team of 5 and cut hospital costs by 18% using Snowflake and Power BI.

5 unverified claim(s):
 - number '5' is not in your fact sheet
 - number '18%' is not in your fact sheet
 - name/tool 'Snowflake' is not in your fact sheet
 - name/tool 'Power' is not in your fact sheet
 - name/tool 'BI' is not in your fact sheet
Verdict: REWRITE before using
```

The first sentence traces to the fact sheet; the second is invented — exactly the failure to catch by hand even without a script. The checker is deliberately simple: it reports "Power BI" as two unknown words and would miss an invented claim with no number or capitalised name ("led the team"), so it supports your reading rather than replacing it. The `FakeLLM` returns a fixed draft so the example runs without a real model. With a real API (the web endpoint a program calls) the prompt is sent like this (needs an **API key** — a secret string identifying your billed account — not run; see Chapter 6):

```python
import anthropic
client = anthropic.Anthropic()                    # reads ANTHROPIC_API_KEY
msg = client.messages.create(
    model="claude-sonnet-5-5",                      # a mid-size model is plenty for short drafts; ids change, check the docs
    max_tokens=300,
    messages=[{"role": "user", "content": prompt}],
)
draft = msg.content[0].text
```

Common mistakes

- Pasting a one-line request and accepting the first answer.
- Letting the model "estimate" a metric you never measured.
- Pasting others' personal data (a manager's email, client figures under an NDA — non-disclosure agreement) into a consumer chatbot. Check its data settings; strip what you don't need.

Self-check

1. Why does an LLM invent a team size? → It predicts plausible text; a plausible number fits the gap.
2. What is a fact sheet? → Your private list of true, checkable facts every draft must trace back to.
3. What should the model write when information is missing? → A placeholder such as `[ASK ME]`, not a guess.

TL;DR

AI improves wording; you own the facts. Facts in → draft out → check every claim → rewrite in your voice.

16.2 The master prompt pattern

What it is

One template for every career document: fill the six slots, drop those that don't apply.

Slot	Question it answers	Example
Role	Who should the model act as?	"Act as a senior recruiter for data roles."
Context	Who are you, what is the situation?	"I'm a junior analyst moving from retail into healthcare."
Facts	What is true?	Bullet list from your fact sheet
Task	What exactly to produce?	"Write a LinkedIn headline."
Constraints	Audience, tone, length, must/must-not	"Under 220 characters, no buzzwords, no invented numbers."
Format	Shape of the output	"Give 5 options as a numbered list."

How it works

1. **Draft with the template**, asking for several versions — comparing three beats perfecting one.
2. **Iterate with short follow-ups**: "shorter", "more formal", "lead with the result", "merge option 2's opening with option 4's ending".
3. **Keep a prompt bank** of prompts that worked; next time swap only the facts.
4. **Get a human read** — a friend or mentor catches tone problems and gaps the model can't know.

TL;DR

Role, context, facts, task, constraints, format — iterate in small steps and save what works.

16.3 Portfolio: sections, project stories, polish

What it is

A **portfolio** is a curated showcase of your work — usually a website, sometimes a PDF or a GitHub profile (GitHub hosts code repositories, project folders tracked with Git) — that *shows* what you can do. Strong portfolios rest on four sections:



Figure — The four core portfolio sections (course diagram)

Section	Purpose	What goes in	Prompt pattern
Professional summary	Who you are in 2–3 sentences	Role or core skill, a distinctive focus, a human tone	"From these facts, write three 40-word summaries for a backend (server-side) developer focused on payments systems."
Projects	Proof of ability	Title and context, *your* part, results and numbers, tools	"Turn these notes into 4 scannable bullets: problem, my part, result, stack."
Testimonials	Social proof	Short quotes with name and role, **used with permission**	"Here is what my former lead wrote to me; suggest a 2-sentence excerpt that keeps her exact meaning."
Contact	Easy to reach you	Professional email, LinkedIn, GitHub, optional booking link (where people pick a meeting slot)	"Write a two-line closing inviting recruiters to get in touch."

A testimonial must come from the named person. Never ask an AI to *create* a quote attributed to someone — that's fabrication. Use AI only to help *request* one ("draft a polite message asking my former manager for a two-sentence recommendation about the migration project") or to suggest an excerpt the person approves.

How it works: project write-ups

Strong project descriptions follow **challenge → solution → result**, then translate numbers into **impact**:

1. **Challenge** — the problem and why it mattered. *"Support tickets waited two days for a first reply."*
2. **Solution** — what *you* did, with tools. *"I built a triage classifier in scikit-learn that routed tickets to the right queue."* (A **classifier** sorts inputs into categories; scikit-learn is a popular Python ML library.)
3. **Result** — a measured outcome. *"First-reply time fell to 6 hours."*
4. **Impact** — what it meant to people or the business. *"Customers got same-day answers, and the team stopped weekend catch-up shifts."*
5. **Link to a bigger goal** — cost, growth, reliability, reputation, where true.

Prompts:

- "Here are my rough notes on the ticket-triage project. Write one paragraph covering the challenge and my solution, for a non-technical reader."
- "Rewrite this result so it explains why it mattered to customers, in two sentences. Keep the numbers exactly."
- "I saved about 15 hours a week for my team. Suggest two ways to phrase the impact without exaggerating."

How it works: polishing

- **Bullets from paragraphs** — "Split this paragraph into three bullets on scope, team and deadline"; for depth, the reverse: "Expand each bullet into two sentences: what I did and what changed."
- **Action verbs** — *built, led, reduced, automated, shipped*.
- **Consistent voice** — first person ("I built…") or third ("Priya built…"), everywhere. Ask: "Check these three sections for mixed person, mixed tense and uneven bullet length; list the fixes."
- **A mini style guide** — your choices (person, tense, date format, tool-name spelling, heading style) pasted into future prompts.

Practice exercises

1. **Portfolio introduction.** "Write a 50-word portfolio introduction. I am a [role] with [N] years in [focus]. Tone: [friendly/formal]. Mention why I care about [specialty]." Generate three, combine the best parts.
2. **Bullets → paragraphs.** Take three real bullets: "Expand each into three sentences: the action, the method or tools, and the result." Check each answers *what, how, why it mattered*, then adjust tone

("more formal", "focus on business impact").

Common mistakes

- Duties ("responsible for testing") instead of outcomes.
- Metrics without meaning — "improved performance by 30%" of what, for whom?
- Showing every project; three to six strong ones beat twenty weak ones.

Self-check

1. The four core sections? → Summary, projects, testimonials, contact.
2. What turns a metric into impact? → Who benefited and how (time, money, satisfaction, reliability).
3. Can AI write a testimonial for you? → It can help you request or trim one; words and permission come from the real person.

TL;DR

Four sections; every project as challenge → solution → result → impact; one consistent voice; real testimonials only.

16.4 Resume: concise, tailored, keyword-aware

What it is

A **resume** (CV in many countries) is a one- or two-page summary of work history, skills and achievements. It does a different job from the portfolio:

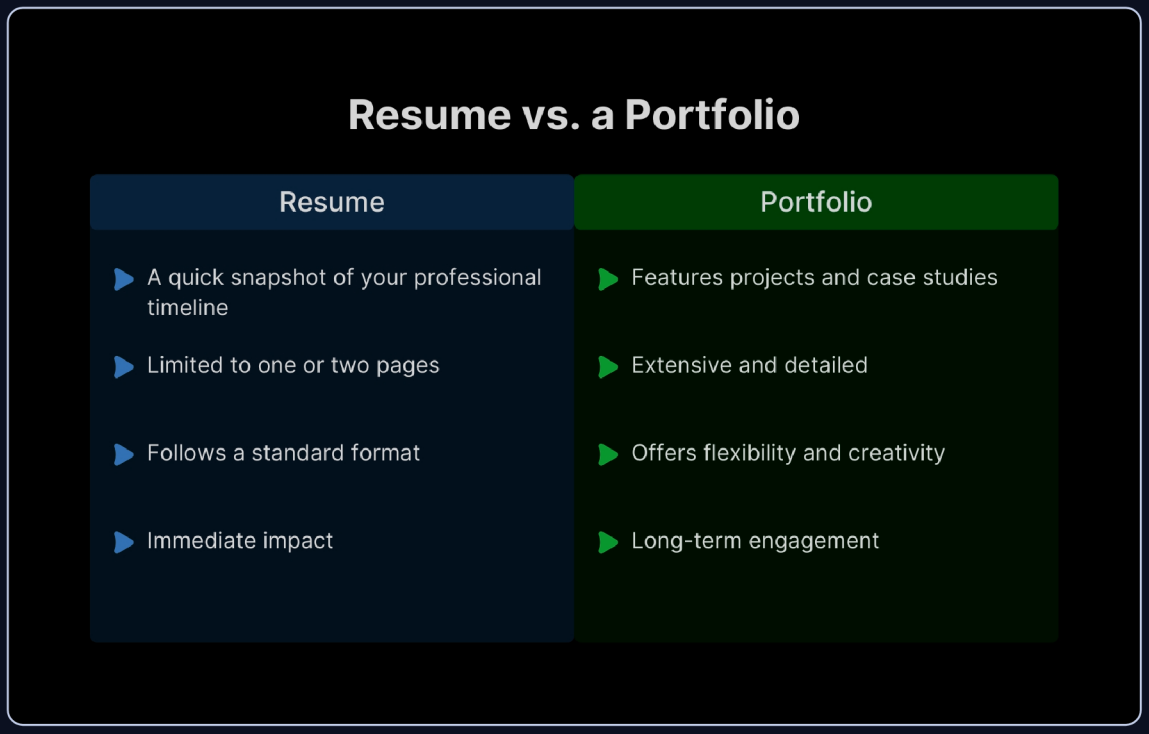


Figure — Resume vs portfolio at a glance (course diagram)

	Resume	Portfolio
Purpose	Quick snapshot: do you meet the bar?	Proof: can you really do it?
Length	1–2 pages, very selective	As deep as needed per project
Format	Conventional (contact, summary, experience, education, skills)	Flexible, creative layout
Reading time	Seconds on a first pass	Explored at the reader's pace
Best for	Applications, networking events	Interviews, client pitches, email signature link

The resume earns a closer look; the portfolio converts it into an interview. Use both.

How it works: section prompts

Section	Goal	Prompt pattern (your facts in brackets)
Summary	Who you are and what you want, 2–3 sentences	"Write a 3-sentence summary for a [role] with [N] years in [domain], strongest in [skills]. Target: [job type]."
Skills	Top technical and people skills	"From my experience below, pick the 8 skills most relevant to [job title]. Group into languages, tools, practices."
Experience	Achievements per role	"Turn these notes about my [title] role into 4 bullets: action verb first, one number each where I gave one."
Education	Degree, certificates, relevant courses	"One line for my [degree] from [university], [year], plus the two courses most relevant to [role]."

Bullet templates

- **Achievement:** "For a [job title], write bullets about [one project]: what I did and the measured result I give you." → *"Cut monthly cloud spend by 22% by right-sizing idle compute and adding autoscaling."* (Right-sizing = moving to smaller servers that still fit the load; **autoscaling** = adding or removing servers automatically with traffic.)
- **Hard + soft skill:** "Write 3 bullets showing my [technical skill] and my [soft skill, e.g. mentoring]." → *"Mentored two new hires through their first on-call rotation while leading the migration to Kubernetes."* (On-call = taking turns being paged when production breaks; Kubernetes = a system for running and scaling containerised applications.)
- **Problem solved:** "Describe how I fixed [problem] at [company]: issue, action, outcome — one line." → *"Replaced a manual invoice-matching step with a Python script, reducing errors from ~40 to ~5 per month."*

(The numbers are illustrations; yours come from your fact sheet.)

How it works: tailoring to a job

An **applicant tracking system (ATS)** is software employers use to receive, store and search applications (Workday, Greenhouse, Lever, iCIMS). Recruiters often filter by keywords and human reviewers scan for the job's terms; how much automatic ranking happens varies by employer and product. The safe goal: a resume that is easy to parse (software can split it into name, jobs, skills — plain headings, no text in images) and uses the job's own words *for skills you really have*.

1. **Keep a base resume** with everything true you might include.
2. **Extract the job's keywords**: "List the 10–15 skills, tools and qualities this posting emphasises, most important first."
3. **Find the gap** — by hand or with the script below.
4. **For each missing keyword: is it true for me?** Yes → add it naturally where it belongs. No → leave it out (or plan to learn it, if it matters).

5. **Tailor**: "Here is my base resume and the job. Reorder and reword to emphasise [themes]. Do not add any skill or number that is not already in my resume."
6. **Review every change**; save a named version per application (e.g. `resume-backend-fintech.pdf`).

`code/keyword_gap.py` extracts the posting's top keywords (known multi-word phrases first, then frequent non-stop-words) and reports which appear in the resume as whole words:

```
def gap_report(job, resume, top=15):
    r = normalise(resume)
    keys = keywords(job, top)
    matched = [k for k in keys
               if re.search(r"(?<![a-z])" + re.escape(k) + r"(?![a-z])", r)]
    missing = [k for k in keys if k not in matched]
    return {"keywords": keys, "matched": matched, "missing": missing,
            "coverage": round(100 * len(matched) / len(keys), 1)}
```

Output:

```
Job keywords : data pipelines, rest apis, unit testing, code review, ci/cd, stakeholder communication, python,
aws, postgresql, backend, docker, kafka
In resume    : rest apis, unit testing, code review, python, aws, postgresql, docker
Not in resume: data pipelines, ci/cd, stakeholder communication, backend, kafka
Coverage     : 58.3%
Next step    : for each missing term, ask 'Is this true for me?' Only true ones get added, in your own words.
```

Coverage = share of the job's keywords found in your resume. (REST APIs = web interfaces programs call over HTTP; Docker packages software into portable containers; AWS = Amazon's cloud; PostgreSQL = an open-source database; Kafka = a system for streaming high volumes of event messages between services.) If you really set up the GitHub Actions pipeline (GitHub's automation running tests on every change) at your last job, "CI/CD" — continuous integration / delivery, automatically testing every change and shipping it once it passes — belongs on the resume. If you've never touched Kafka, it doesn't — "a plus" in a posting is not a requirement.

Common mistakes

- **Keyword stuffing** — long term lists or white-on-white text to game a filter. Humans notice; it reads as dishonest.
- Letting the model "upgrade" your title or scope ("contributed to" becoming "led").
- One generic resume for every job.
- Dates and titles not matching your LinkedIn and portfolio.

Self-check

1. Resume vs portfolio in one line? → Resume = fast snapshot; portfolio = detailed proof.
2. ATS? → Software that stores and searches applications; recruiters filter by keywords in it.
3. The posting wants Kafka; you've never used it? → Leave it out; learn it and mention it only once true.

TL;DR

Base resume + tailored versions; job keywords only where true; bullets = verb + action + result; verify every AI edit.

16.5 Cover letters

What it is

A **cover letter** is a short letter sent with your application explaining why *this* job at *this* company, and why you — often the employer's first sample of your writing. It shows you researched the company (generic signals low effort); it's the one place to explain a career change, gap or unusual path

positively; it reveals communication skill and personality, which can decide between equal candidates; and errors in it suggest carelessness.

How it works: structure

1. **Header** — name, email, phone, LinkedIn or portfolio link.
2. **Greeting** — a named person where possible ("Dear Ms. Okafor"); find the hiring manager on the company site or LinkedIn.
3. **Opening** — the role and one or two sharp reasons you fit; optionally something specific you respect about the company.
4. **Body (1–2 paragraphs)** — one or two short work stories matching the job's main needs.
5. **Closing** — restate interest, invite a conversation, thank them.
6. **Sign-off** — your name, plus a portfolio link if not in the header.

Aim for **250–350 words**; readers skim.

How it works: prompting

1. **Research first** — the company's stated values, a recent product or project, the job's top requirements. The model can't know last month's announcement unless you paste it.
2. **Draft**: "Write a cover letter for a [role] at [company]. Their values: [pasted]. Job priorities: [pasted]. My matching facts: [fact sheet]. Tone: [formal for a bank, warmer for a startup]. 300 words."
3. **Inject yourself**: "Add a short paragraph about [a real experience, e.g. volunteering as a coding tutor] and link it to their mission." Then ask *does this sound like me?*
4. **Adjust in small steps**: "Make paragraph two more enthusiastic but still professional." "Cut to two paragraphs without losing the migration story."
5. **Customise per job** — tone (corporate vs startup), length, highlighted projects ("Emphasise my SQL and dashboard work; this role is reporting-heavy" — SQL is the standard database query language; a dashboard is a page of live charts and numbers).
6. **Proofread** — a model grammar pass, then a human read.

Common mistakes

Mistake	Better
"To Whom It May Concern"	A named person, or at least "Dear [Team] Hiring Team"
Unrelated hobbies and life story	Only details that connect to the role or company
Claiming qualities ("passionate team player")	Showing them with a specific example
Same letter to every company	One paragraph that could only be written for this company
No proofreading	AI grammar pass **and** a human read
AI-sounding filler ("I am thrilled to leverage my synergies…")	Plain, specific sentences

Self-check

1. Ideal length? → About 250–350 words.
2. What can a cover letter do that a resume can't? → Explain motivation, career changes or gaps; show voice and fit.
3. Why paste company research into the prompt? → The model may not know recent or specific facts; without them it writes generic flattery.

TL;DR

Header → greeting → hook → one or two matching stories → call to action. Research in, specifics out, your voice on top.

16.6 Networking and job-search emails

What it is

Four kinds of email drive most job-search networking:



Figure — The four common job-search email types (course diagram)

Type	When	Must contain
Cold outreach	Contacting someone you don't know: recruiter, hiring manager, potential mentor	Short clear subject; who you are; why *them* (something specific about their work); one small, clear request
Follow-up	After applying (no reply in 1–2 weeks) or after meeting someone	Reminder of the last contact; purpose; polite, no pressure
Referral	Someone suggested you contact a person, or you ask a mutual contact to recommend you	Name the referrer first; why you fit; thanks to both
Thank-you	After an interview, advice or an introduction	Thanks; one specific thing discussed; offer to help or stay in touch

How it works: structure with AI help

1. **Subject line** — short (under ~60 characters so most screens don't cut it), specific. "Suggest 5 subject lines for an email asking a data engineer at [company] for a 15-minute chat about their team's migration to dbt" (dbt: a tool for writing and testing SQL data transformations), then "Make these less formal". Pick yourself — the model can't know real open rates (share of recipients who open), so any "this will perform best" is a guess.
2. **Opening (1–2 sentences)** — who you are and why you're writing, from the reader's view: "Write a two-sentence opener to [name] that mentions her conference talk on [topic] and says why I'm writing."
3. **Body** — at most two short paragraphs; one ask.
4. **Closing** — a clear call to action ("Would a 15-minute call next week work?") and sincere thanks.

5. **Keep it short** — two or three brief paragraphs; busy inboxes skim.

Prompt templates

- **Informational interview** (an informal chat to learn about someone's job or field, not a job interview): "Help me write a short, friendly-professional email to [name], [title] at [company]. I'm a [your status] interested in [topic]. Mention [specific thing about their work]. Request a short (about 15 minutes) chat on how they got into [field]."
- **Application follow-up:** "Write a brief follow-up to the hiring manager at [company] about my [position] application sent on [date]. Restate interest, name one skill I bring ([skill]), offer more information. Polite, not pushy."
- **Thank-you after an interview:** "Write a thank-you email to [name] after my [position] interview. Mention our discussion about [specific topic], show enthusiasm for next steps, offer to send anything else they need." Send within about 24 hours.
- **Referral:** "Write a short email to the hiring manager at [company]. [Referrer], who works on their [team], suggested I contact them about the [role]. Include my two most relevant facts: [facts]."
- **LinkedIn connection note:** "Write a connection note under 200 characters to [name] mentioning [shared event or interest]." (In 2026 LinkedIn allows about 200 characters on free accounts and 300 on Premium, its paid tier; free accounts get only a limited number of personalised notes per month — check current limits.)

Common mistakes

Pitfall	Why it hurts	Fix
Tone mismatch	Too casual reads as unprofessional; too stiff as cold	Match the recipient (startup founder vs corporate HR); ask for "friendlier" or "more formal" and choose
Grammar and typos	Signals carelessness	Proofread; read aloud; a grammar checker as second pass
Robotic AI phrasing	Formulaic lines feel mass-produced and reduce trust	Add a real detail in your words; replace stock phrases; ask for "more personal, less formal"
Following up too soon or too often	Feels pushy	Wait about a week; one or two polite follow-ups at most
Mass-sending one template	Recipients recognise it	At least one sentence only this person could receive

Self-check

1. Four email types? → Cold outreach, follow-up, referral, thank-you.
2. What goes first in a referral email? → The referrer's name.
3. Interview thank-you timing? → Within about 24 hours.

TL;DR

Short subject, reader-centred opening, one ask, clear close, a real personal detail. Templates speed you up; specifics get replies.

16.7 LinkedIn

What it is

LinkedIn is a professional network and, in practice, a public portfolio page that ranks high in searches for your name. Three purposes: **networking** (peers, mentors, groups, companies), **job hunting** (listings, applying with your profile, recruiters searching candidates) and **personal branding** (posts,

articles, a Featured section pinning your best work). Unlike a resume, it updates any time and holds links, documents and media.

How it works: the five key sections

Section	Job	Tips	Prompt pattern
Headline (up to 220 characters)	The line under your name, shown in search results	Role + specialty + value in words recruiters search; readable, not a keyword pile	"Give 5 headline options for a frontend developer focused on accessibility and design systems. Under 120 characters each." (Frontend = the browser-side part users see; accessibility = usable by people with disabilities; design system = a shared library of UI components and style rules.)
About (up to 2,600 characters)	Your story: what you do, how, what next	Hook in the first two lines (only they show before "see more"); two or three real achievements; some personality	"From these facts, draft a warm, first-person About section: what I do, two results, what I'm looking for. 200 words."
Experience	Roles with outcomes	Bullets, results over duties, defensible numbers	"Turn these notes on my product manager role into 3 bullets on research, launch and metrics. Use only my numbers."
Featured	Pinned work at the top	Only your best; mixed formats (article, code repository, demo video, PDF case study); a short caption each	"Write a 1-sentence caption for my Featured case study on reducing checkout drop-off."
Skills & endorsements	Lets search match you; endorsements add proof	Most relevant first; specific ("Agile project management" — short iterative cycles — not "management"); ask colleagues to endorse skills they saw	"Suggest 10 specific skills a cloud engineer with AWS, Kubernetes and Terraform (creates cloud infrastructure from configuration files) should list, most searched first." Keep only the true ones.

How it works: links and media

- **Featured** — links, documents (PDF case studies, slides) or media, each with a clear title and one-line takeaway, strongest first. Menus move between app versions: look for *Add profile section* or the section's edit pencil.
- **Projects** — name, your contribution, tools, a result, a link, teammates if relevant.
- **Personal website** — say what visitors will find; a custom domain (e.g. `yourname.dev`) looks polished; keep it current.
- **GitHub** — a handful of representative repositories, each with a clear README (the repository's front-page file: what it does, how to run it, your role).
- **Do:** high-quality media, a short story per item, best first. **Don't:** dozens of links, empty descriptions, stale projects.

How it works: staying current and active

1. **Audit every few months** (or after a big project): paste your sections and ask, "Review this for clarity and consistency. What looks outdated or missing for a [field] professional?" Treat "latest trends" remarks as hints to verify.
2. **Engage** — thoughtful comments in your field; share what you built or learned.
3. **Connect with a note** saying why (a shared event, a talk you liked).
4. **Join groups and events** in your field.
5. **Post about finished work**: "Draft a short LinkedIn post announcing my [project], its main result, and asking designers for feedback." Edit until it sounds like you.

Common mistakes

Headline that is only a job title ("Software Engineer at X"); About in stiff third person or pure buzzwords; dates and titles disagreeing with your resume; collecting connections without ever talking to them.

Self-check

1. Why does About's first line matter most? → Only the opening lines show before the reader expands.
2. A good Featured item? → Your best, most relevant work with a one-line takeaway.
3. AI-suggested skills? → A checklist: keep the true ones, discard the rest.

TL;DR

Keyword-aware headline, human About, outcome bullets, a few pinned best pieces, true skills — plus regular audits and activity.

16.8 Freelancing: marketplaces, proposals, link hubs

What it is

Freelance **marketplaces** connect independent workers with clients, each with its own audience: **Upwork** (clients post jobs, freelancers send proposals), **Fiverr** (freelancers list fixed-price "gigs" buyers pick), **Toptal** (a screened network for senior talent), plus niche sites for design, writing or specific tech. Pick the one whose clients and projects match your skills.

How it works: proposals that convert

1. **Read like the client**: "Analyse this job post. List the client's main problem, the outcome they want, and any unstated worries." Answer *those*, not your life story.
2. **Hook in the first line** on their need: "Write three opening sentences for a proposal to speed up a slow Shopify store" (Shopify: a hosted online-shop platform).
3. **Only relevant proof** — one or two similar projects with results: "Turn these bullets from my past ad campaign into a 3-sentence mini case study."
4. **A short plan** — 3–4 steps (e.g. audit → fix the biggest bottlenecks → measure → iterate): "Outline a plan for this redesign in under 120 words."
5. **Why you** — one distinctive strength.
6. **Call to action** — "Happy to do a 15-minute call to review your current setup."

How it works: researching the market

A model can say what top profiles in your category usually emphasise and summarise strong bios, but it **can't see live marketplace data** without a browsing tool, and its training data may be old. For rates and demand, read real profiles and job posts, or paste a set of posts and ask for recurring requirements and keywords — then build your profile around what clients actually ask for.

How it works: one link for everything

- A **link-in-bio page** (Linktree and similar) gives one URL listing your profiles — Upwork, Fiverr, portfolio, GitHub, LinkedIn — with descriptive labels ("See my case studies", "Hire me on Upwork"), short and ordered by importance; put it in your email signature and social bios.
- A **personal website** gives room for case studies, testimonials, a blog, and search visibility. Keep branding (colours, fonts, photo, bio) consistent with your other profiles; lead with your niche.
- Prompts: "Write a 100-word homepage intro for a full-stack developer (browser and server side) specialising in e-commerce performance." "Describe my three services in one sentence each, focusing on the client's benefit." "Suggest five short call-to-action buttons for my site."

Common mistakes

- Copy-paste proposals that ignore the client's post; listing every skill instead of the relevant ones.
- Trusting a chatbot's "average rate" without checking live data.
- Ignoring platform costs: marketplaces take a service fee, and some (e.g. Upwork) charge credits to send proposals — read the current fee page before pricing.

TL;DR

Client's problem first, relevant proof, a short plan, a clear next step. Research real listings; one link hub ties your profiles together.

16.9 Job-hunt automation and search keywords

What it is

Two time-saving uses: **filtering** listings against your criteria, and choosing **keywords** — the words recruiters and search engines use — so people find you. **SEO (search engine optimisation)** is wording content so search tools rank it for the right queries; here, for your portfolio, resume and LinkedIn.

How it works: filtering listings

A plain chatbot doesn't search live postings without a browsing tool; it works on what you give it.

1. **Collect postings** — paste (or export) the listings into one document.
2. **Filter**: "Here are 20 postings. Which require Python and SQL? Show title, company and location as a table."
3. **Narrow**: "Of those, which mention remote work and a salary of at least [amount]?"
4. **Match**: "My background: [fact sheet summary]; I want [remote, data-heavy, healthcare]. Score each role 1–10 for fit and explain the score in one line."
5. **Summarise the top three**: requirements, stated salary range, culture signals.
6. **Verify** — open the real posting; check it's current and the model didn't misread requirements.

A model's fit score is a rough sorting aid, not a judgement — it can miss a dealbreaker buried in the text.

How it works: keywords

1. **List your terms** — role titles ("data analyst"), tools ("Power BI" — Microsoft's dashboard tool — "dbt"), methods ("A/B testing", comparing two versions on randomly split users), broad skills ("stakeholder management").
2. **Ask for more**: "List 10 terms recruiters search for when hiring a mid-level React and TypeScript developer" (React: a JavaScript library for web interfaces; TypeScript: JavaScript with type checking). Narrow by level, industry or method; run twice and keep terms that recur.
3. **Cross-check with real postings** — several current ads using a term means it matters; a model's "trending skills this year" reflects possibly stale training data.
4. **Place them where they count** — portfolio titles, headings, first sentences; resume skills and summary; LinkedIn headline, About and Experience; specialty in job titles where accurate ("Software Engineer — Backend").
5. **Be specific** — "Scrum" (an Agile method of fixed-length sprints) or "Agile project management", not "management"; group related skills.
6. **Keep it readable** — "Built a React and Node.js checkout that loads 40% faster" (Node.js runs JavaScript on servers) beats a keyword list; stuffing hurts with humans and search alike.
7. **Refresh** every few months as your field's language changes — only with terms true for you.

Self-check

1. Can a plain chatbot find live postings? → Not without a browsing or search tool; paste them in.

2. Top keywords on LinkedIn go where? → Headline, About, Experience (and Skills).
3. Validate a "trending" keyword how? → Check multiple current postings use it.

TL;DR

Paste listings in, filter and score, verify. Keywords from real postings, in high-value spots, in natural sentences.

16.10 Interview preparation

What it is

A chatbot as practice partner: it plays the interviewer, sets problems at your level, critiques answers and explains alternatives — any hour, as often as you like.

How it works: technical practice

1. **Set the scene**: "You are interviewing me for an entry-level Python job. Set one problem I can finish in 30 minutes, on lists and dictionaries. Don't show the solution until I ask."
2. **Cover the core topics** — patterns (two pointers, sliding windows, recursion), algorithms (binary search, sorting, dynamic programming) and structures (arrays, hash maps, stacks, queues, trees, graphs): "Give me a medium tree problem that needs depth-first search, with a sample input and expected output."

   > **Key term — the classic interview toolkit:** *Two pointers* walks two indexes through a list (often from both ends) instead of nesting loops; a *sliding window* keeps a moving range updated one step at a time; *recursion* is a function calling itself on a smaller piece. *Binary search* halves a sorted range each step; *dynamic programming* stores answers to overlapping sub-problems so each is computed once. A *hash map* (dictionary) looks values up by key in about constant time; a *stack* is last-in-first-out, a *queue* first-in-first-out; *trees* and *graphs* are nodes joined by links, and *depth-first search* explores one branch fully before backing up.

3. **Debugging drills**: "Give me a short function with one subtle bug. I'll find it; then show the fix."
4. **Discuss after solving**: "My solution is O(n²). Is there an O(n log n) or O(n) approach? Compare time and space." (**Big-O notation**: how time or memory grows with input size n — O(n²) means doubling the input roughly quadruples the work.)
5. **Scenario questions** (system design, troubleshooting): "Act as a senior engineer. Our API's latency (time per request, start to finish) tripled after a release — ask me how I'd find the bottleneck, one question at a time." "Traffic spikes 10× on sale days; interview me on how I'd scale it." (Building blocks — latency, throughput, caching, load balancing: System Design Fundamentals; AI-specific design interviews: Chapter 13.)
6. **Mix in soft skills**: "During the scaling discussion, sometimes ask how I'd explain the change to non-technical stakeholders."

Tips: a real timer; **reason out loud** (interviewers score the thinking, not just the code); log where you got stuck and ask about those gaps; include hard problems. Check the model's answers — it can be wrong about complexity or write buggy code, so run it.

How it works: behavioural answers with STAR

Behavioural questions ("Tell me about a time you…") test how you work with people and problems. **STAR** structures the answer:



Figure — The STAR structure for behavioural answers (course diagram)

Step	Content	Example
Situation	Context in one or two sentences	"Two weeks before a client launch, our payment provider changed its API."
Task	Your responsibility	"I owned the integration and had to keep the launch date."
Action	What *you* did (most of the answer)	"I split the work into two parallel streams, wrote contract tests (automated checks that the API still sends and accepts the agreed data shapes) against the new API, and gave the client daily status notes."
Result	Outcome, measured if possible, plus the lesson	"We launched on time with zero payment failures in the first month; we now run contract tests for every provider."

Pick stories matching the job (collaboration-heavy role → a collaboration story); prepare 6–8 that flex across common questions (conflict, failure, leadership, tight deadline, ambiguity). With AI:

- "Here is my STAR answer about a difficult stakeholder. Critique clarity and length; point out where the Action is vague."
- "Rewrite it under 200 words and end on the result." / "Make it more conversational but still professional."
- Then **check it still describes what really happened**, restoring specifics the rewrite smoothed away.

How it works: after the interview

1. **Thank-you email within 24 hours** — two or three short paragraphs, one specific reference to the conversation, a brief restatement of fit, an offer to answer further questions.
2. **Reflect while fresh** — what went well, surprising questions, unclear explanations, technical or behavioural gaps.
3. **Plan** one or two improvements; ask for targeted practice ("Give me five behavioural questions about handling scope creep" — requirements growing beyond what was agreed).
4. **Ask for feedback** from the recruiter or a mentor where possible.
5. **Track progress** across interviews and adjust preparation.

Ethics note

Practising with AI is fine. Using AI *during* a live interview or take-home test without permission is different: many employers forbid it, some ask candidates not to use AI in applications at all. If

unclear, ask. Being caught misrepresenting your ability usually ends the process — and passing that way lands you a job you can't yet do.

Self-check

1. STAR? → Situation, Task, Action, Result.
2. Longest part? → Action — what *you* did.
3. Why speak aloud while practising? → Interviewers assess reasoning; saying it builds clarity.

TL;DR

Role-play problems at your level, time yourself, think aloud, verify AI answers. STAR for stories. Thank-you within a day, then reflect and target weak spots.

16.11 Personal brand, consistency, reputation

What it is

Your **personal brand** is what people consistently associate with you professionally — focus, values, way of working — across every place they find you.

How it works: brand voice

1. **Name your values**: "Suggest ten adjectives that fit a data engineer who cares about reliability, clear documentation and mentoring." Keep the three or four that are really you.
2. **Write a brand statement**: "Using these words, write three one-line brand statements I could adapt for a LinkedIn headline." Adjust tone ("friendlier, still professional").
3. **Stay authentic**: include challenges and lessons, not only wins; evidence over vague claims — not "passionate about open source" but "merged 30 pull requests into two open-source libraries this year" (if true; a **pull request** is a proposed code change for review; **open source** code is public for anyone to use and improve); highlight what makes you different (a teacher turned developer who writes unusually clear docs); edit AI output until it sounds like you.

How it works: consistency across platforms

Recruiters check several places; make them tell one story.

Platform	Keep aligned
LinkedIn	Headline and About echo your portfolio bio; Featured shows the same flagship projects; endorsed skills match those you emphasise
GitHub	Repository names match portfolio project names; READMEs describe the goal and your part; pin the projects your LinkedIn mentions
Website / blog	Same photo, colours and bio; links to LinkedIn and GitHub and back
Everywhere	Same dates, titles and numbers — "three years leading an Agile team" must match your work history

Prompt: "Here are my LinkedIn About, GitHub bio and website intro. List any contradictions in dates, titles or claims, and any differences in tone."

How it works: reputation management

1. **Monitor** — alerts on your name and project names (Google Alerts is free); check comments, issues and messages.
2. **Classify** — positive, neutral, question or complaint.
3. **Respond promptly and politely** — if you need time, say so and follow up.

4. **AI for a calm first draft**: "Draft a friendly reply to a user who liked my library but said the docs are thin. Thank them and say what I'll add." / "Draft a short, empathetic reply to someone who found my app's signup confusing. Under 80 words, no defensiveness."
5. **Personalise, own mistakes, keep it short**; after fixing a public issue, post a short note so others see the resolution.

How it works: growing your reach

- **Thought leadership** — a blog (own site or Medium) for long posts, LinkedIn for shorter; one or two pieces a month (project write-ups, lessons, commentary). AI for outlines and editing ("Suggest an outline for a 600-word post on what I learned migrating to PostgreSQL 16"), but the experience and opinions must be yours. Reply to comments.
- **Collaboration** — partner with complementary skills (developer + designer), co-write posts, run a joint webinar (online live talk), guest on a podcast: "Draft a short email inviting a UX (user experience) designer I met at a meetup to co-write a post on accessible forms."
- **Events and communities** — meetups (small local gatherings), conferences (offer a talk), field-specific Slack or Discord groups where you answer questions.
- Networking works when mutual: offer help, not just requests.

TL;DR

A few true values, expressed consistently everywhere; evidence over adjectives; calm, prompt, honest public responses; share what you learn.

16.12 Keeping it current, and a checklist

How it works: scheduled audits

1. **Pick a rhythm** — monthly if your work changes fast, else quarterly — in your calendar.
2. **Content check**: "Review my portfolio homepage and About text for clarity, consistency and anything that sounds dated."
3. **Structure check**: "Here is my site's page list and navigation. Suggest how to make my recent analytics projects easier to find."
4. **Broken links** — a link-checker tool or browser extension; a chatbot can't click links but can spot suspicious ones in a pasted list (renamed services, old URL formats).
5. **Stale content** — update old roles and skills; replace "in progress" with outcomes; refresh metrics and screenshots; archive projects that no longer represent you.
6. **Change gradually** and log what changed, to see what improved responses.

How it works: tracking your field

Ask a model to summarise emerging roles, skills or tools — then **confirm with current sources** (job boards, engineering blogs, industry reports). Models have a **knowledge cutoff** (no training data after a date) and aren't live; their salary comparisons are broad impressions at best. Follow trends matching your goals, and when you learn a new skill, add a portfolio line on why and how you used it.

Quick checklist

Area	Done when…
Fact sheet	All roles, dates, tools, projects and numbers written down with sources
Portfolio	Summary; 3–6 projects told as challenge → solution → result; real testimonials with permission; clear contact
Resume	Base version plus tailored copies; keywords only where true; dates match everything else
Cover letters	Researched, specific paragraph per company; 250–350 words; proofread by a human
Emails	Templates for outreach, follow-up, referral, thank-you — each personalised before sending
LinkedIn	Headline, About, Experience outcomes, Featured best work, true skills
Freelance	Profile matches portfolio; proposal template starting from the client's problem; one link hub
Interviews	Timed technical practice; 6–8 STAR stories; thank-you within 24 h; reflection notes
Maintenance	Calendar audits; link check; trends verified against real postings

Next steps

- Keep a **prompt bank**; try new assistant features (file upload, code review, voice practice) as they appear.
- Practise different prompt types — brainstorming, style editing, fact-checking your own claims.
- Show drafts to people; fold their feedback into your next prompts.
- Keep learning your field (courses, certificates, projects) and show in your portfolio how you adapt — including how you use AI responsibly.
- Build real relationships: meetups, forums, open-source contributions.

16.13 Ethics and honesty (read this twice)

Rule	Why
Never fabricate experience, employers, titles, dates, degrees, certifications, metrics, or testimonials	Dishonest; background checks, reference calls and technical interviews expose it; offers can be withdrawn and employment ended for falsified applications
Don't inflate scope ("helped with" ≠ "led")	Interviewers ask follow-ups; you must defend every line in detail
Numbers need a source	If you can't explain how it was measured, soften it ("roughly halved") or drop it
Follow each employer's AI rules	Some forbid AI in applications, take-homes or live interviews; ask when unclear
Protect privacy	Don't paste confidential employer data, client figures under NDA, or others' contact details into consumer tools; check data-use settings
Keep it your voice	Heavily generic AI text reads as low effort and erodes trust in outreach
Watch for bias (systematic unfairness toward some groups)	Models can reproduce stereotypes (gendered wording, assumptions about age or background); review tone and word choice
Don't rely on AI "detectors" or "ATS scores"	Both (tools claiming to spot AI-written text; match scores some ATS tools show) are unreliable; write clearly for humans

Test before sending anything: *could I explain every sentence, with specifics, to the person who will read it?* If not, rewrite it.

Chapter TL;DR

- **Fact sheet first**; give the model only true facts, ask for `[ASK ME]` placeholders, check every number and name (the `claim_check.py` idea), then rewrite in your voice.
- **Master prompt**: role, context, facts, task, constraints, format; ask for several versions; iterate in small steps; keep a prompt bank.
- **Portfolio**: summary, projects, testimonials (real, with permission), contact; projects as challenge → solution → result → impact; one consistent voice.
- **Resume**: quick snapshot vs portfolio's proof; section prompts; verb + action + result bullets; tailor per job using its keywords **only where true** (`keyword_gap.py`).
- **Cover letter**: header, named greeting, hook, one or two matching stories, call to action; 250–350 words; research in, specifics out.
- **Emails**: cold outreach, follow-up, referral, thank-you; short subject, one ask, a personal detail; thank-you within 24 hours.
- **LinkedIn**: headline (≤220 chars), About (≤2,600 chars, strong first lines), outcome bullets, Featured best work, true skills; audit and stay active.
- **Freelance**: client's problem first, relevant proof, short plan, call to action; research on live listings; one link hub.
- **Automation & keywords**: paste listings to filter and score, then verify; keywords from real postings, placed naturally.
- **Interviews**: timed practice, think aloud, verify AI answers; STAR stories; reflect and target weak spots; no undisclosed AI during interviews.
- **Brand**: true values, consistent across platforms, evidence over adjectives, calm public responses, share what you learn.
- **Above all**: AI changes how you say it, never what is true.